SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	Tharun
Register Number	1924260280
Course Code & Name	C Programming
Slot	B
Assignment / Task Title	Logic Analysis and Design Challenges (Q1–Q7)
GitHub Repository URL	
Submission Date	09-11-2026

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

TASK 1 — Precedence, Associativity and the Discount Bug

Task / Problem Statement

A government pharmacy (Jan Aushadhi) must compute the amount payable as: price × quantity, less 15% discount for senior citizens, plus 5% GST. The developer wrote everything as one expression without brackets and customers report wrong amounts:

```
total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
net = total > 1000 ? total - 50 : total;   /* Rs 50 off above Rs 1000 */
count = count++ + 1;
```

The program must predict the result of the expressions by hand using the C precedence and associativity rules, run them, find every expression whose result differs from what the pharmacist intended (including the effect of integer division), explain why the third statement is undefined behaviour, and rewrite the calculation correctly.

Aim

To understand how C evaluates an expression with mixed operators (precedence, associativity, integer division), to locate the bugs in the billing expression, and to redesign it so that it is correct and readable.

Concept / Theory

Precedence decides which operator is applied first: `*`, `/` and `%` bind tighter than `+` and `-`, relational operators such as `>` bind tighter than the conditional `?:`, and `=` is almost the lowest. **Associativity** decides the order between operators of the same level: `*` and `/` group left to right, so `a * b / c` is `(a * b) / c`, while `?:` and `=` group right to left.

With **integer division** the fractional part is thrown away (`299.7` becomes `299`), so the position of `/ 100` in the expression matters. A **sequence point** is a place where all earlier side effects must be finished. Modifying the same variable twice between two sequence points (`count = count++ + 1`) is **undefined behaviour**: the C standard does not say what the result is.

Procedure / Algorithm

1. Take three bills: (price 120, qty 3, senior), (price 450, qty 2, not senior) and (price 999, qty 2, senior).
2. Before running anything, apply precedence and associativity by hand and predict `total` and `net` for each bill.
3. Write the original one-line expression as a function and run it for the three bills.
4. Write the corrected version using intermediate variables `gross`, `discount`, `taxable`, `gst` and compare both results.
5. Show the effect of integer division (`15/100` evaluated first gives 0) and compile `count = count++ + 1` with `-Wall`.

Program / Source Code

```c
#include <stdio.h>

/* Original (buggy) one-line expression, wrapped in a function */
int buggy_total(int price, int qty, int senior)
{
    return price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
}

/* Corrected version: intermediate variables + brackets */
double fixed_total(int price, int qty, int senior)
{
    double gross    = (double)price * qty;
    double discount = senior ? gross * 0.15 : 0.0;
    double taxable  = gross - discount;          /* GST on discounted amount */
    double gst      = taxable * 0.05;
    return taxable + gst;
}

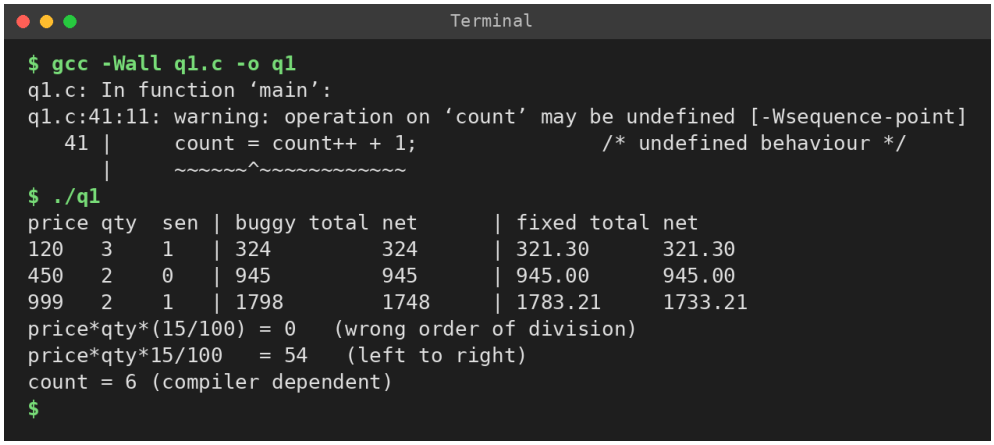
int main(void)
{
    int price[] = {120, 450, 999};
    int qty[]   = {  3,   2,   2};
    int senior[]= {  1,   0,   1};

    printf("%-5s %-4s %-3s | %-11s %-8s | %-11s %-8s\n",
           "price", "qty", "sen", "buggy total", "net", "fixed total", "net");
    for (int i = 0; i < 3; i++) {
        int    t  = buggy_total(price[i], qty[i], senior[i]);
        int    n  = t > 1000 ? t - 50 : t;
        double ft = fixed_total(price[i], qty[i], senior[i]);
        double fn = ft > 1000 ? ft - 50 : ft;
        printf("%-5d %-4d %-3d | %-11d %-8d | %-11.2f %-8.2f\n",
               price[i], qty[i], senior[i], t, n, ft, fn);
    }

    /* Integer-division effect: 15/100 evaluated first would give 0 */
    printf("price*qty*(15/100) = %d   (wrong order of division)\n", 120 * 3 * (15 / 100));
    printf("price*qty*15/100   = %d   (left to right)\n", 120 * 3 * 15 / 100);

    int count = 5;
    count = count++ + 1;               /* undefined behaviour */
    printf("count = %d (compiler dependent)\n", count);
    return 0;
}
```

Output Screenshot



Analysis / Discussion

Prediction (before running). The expression is evaluated left to right inside each term, and `*` and `/` before `+` and `-`. Bill 1: `120*3 = 360`; `360*15 = 5400`, `/100 = 54`, `*1 = 54`; `360*5 = 1800`, `/100 = 18`; so `360 - 54 + 18 = 324`, net = 324. Bill 2: `900 - 0 + 45 = 945`, net = 945. Bill 3: `1998 - 299 + 99 = 1798` (since `1998*15/100 = 299.7` is truncated to 299 and `1998*5/100 = 99.9` to 99), and net = `1798 - 50 = 1748` because 1798 > 1000. The program printed exactly these values, so the predictions were correct.

Price, qty, senior	Original total / net	Intended total / net	Difference
120, 3, yes	324 / 324	321.30 / 321.30	+2.70 over-charged
450, 2, no	945 / 945	945.00 / 945.00	none
999, 2, yes	1798 / 1748	1783.21 / 1733.21	+14.79 over-charged

Expressions that differ from the pharmacist's intention. (1) GST is calculated on the full price `price*qty*5/100` instead of on the discounted amount, so senior citizens pay too much tax. (2) Each percentage is truncated by integer division (299.7 → 299, 99.9 → 99), which loses up to Rs 1 per term. (3) Moving the division first, `price*qty*(15/100)`, would make the discount 0 because `15/100 = 0` (the program printed 0 against 54 for the correct left-to-right order). (4) `net = total > 1000 ? total - 50 : total` is parsed correctly as `(total > 1000) ? (total - 50) : total`, so that line is right, but it inherits the wrong `total`. (5) `count = count++ + 1` is wrong in a different way: it is undefined.

Inquiry – why the third statement is undefined. `count++` modifies `count` and the assignment `=` modifies it again, and there is no sequence point between the two modifications. The compiler is free to produce 6, 5 or anything else. With `-Wall` GCC prints `warning: operation on 'count' may be undefined [-Wsequence-point]` (visible in the screenshot). The fix is simply `count = count + 1;` or `count++;`.

Design decision. I use both brackets and intermediate variables: variables `gross`, `discount`, `taxable`, `gst` give every step a name and let me print or test them separately, and brackets are used only where the order must be forced. The discount is applied first and GST is charged on the discounted amount. In real billing code the amounts should be kept in integer paise, or rounded once at the end, to avoid float rounding.

Alternative concept – intermediate variables. Replacing the single long expression by several statements does not change the time: both do about 8–10 arithmetic operations, O(1) per bill (the temporaries even save repeating `price*qty` three times, although an optimising compiler already does that). The extra space is three or four variables, O(1) and a few bytes on the stack. Correctness and readability improve a lot because each line can be checked on its own.

Result

The original expression was evaluated exactly as predicted by the precedence rules, but it over-charges senior citizens (GST on the undiscounted price) and loses paise to integer division; the statement `count = count++ + 1` is undefined behaviour. The corrected, step-by-step calculation gives the intended amounts. I learned that precedence only tells the order of operators, not the meaning the programmer wants, so long expressions should be split with brackets and named variables.

TASK 2 — When Does Short-Circuit Evaluation Matter?

Task / Problem Statement

A railway ticket counter grants a concession if the passenger is a senior citizen **and** the ticket class is eligible, **or** the passenger holds a disability card. Checking the card calls a slow function that contacts a server. Design experiments to show (a) when the second operand of `&&` and `||` is not evaluated, (b) how the order of conditions changes the number of calls to the slow function, and (c) the difference between `&` and `&&` on conditions. Count the slow calls for 1,000 sample passengers with different orders, then decide when the cheapest or most-likely-false condition should be first. Finally re-evaluate the order when the slow function also writes an audit log.

Aim

To demonstrate short-circuit evaluation of `&&` and `||`, measure how condition order changes the number of slow function calls, and justify the best order for the original and the changed (audit-log) requirement.

Concept / Theory

Short-circuit evaluation: in `A && B`, if `A` is false the result is already false, so `B` is not evaluated; in `A || B`, if `A` is true, `B` is skipped. The operands are evaluated strictly left to right, with a sequence point after the first. The bitwise operators `&` and `|` have no such rule: both operands are always evaluated and in unspecified order. `&&` has higher precedence than `||`, so `(senior && eligible) || card` needs no brackets, but they make the intention clear.

Because a skipped operand is never run, any side effect inside it (a function call, a counter, a log entry) is also skipped. This is useful for saving time but dangerous when the side effect is needed.

Procedure / Algorithm

1. Create 1,000 passengers with a fixed random seed: 20% senior, 60% eligible class, 10% disability card.
2. Show with a tracing function which operands are evaluated for `0 && f()`, `1 || f()` and `0 & f()`.
3. Count slow calls for order A `(senior && eligible) || verify`, order B `verify || (senior && eligible)`, the non-short-circuit version using `&` and `|`, and nested ifs.
4. Change the slow function so that it records each passenger it checks in an audit log and repeat the count.
5. Compare calls, number of concessions granted (must be equal) and audit-log completeness, then decide the best order.

Program / Source Code

```
#include <stdio.h>
#include <stdlib.h>

#define N 1000
typedef struct { int senior, eligible, card; } Passenger;
static Passenger p[N];
static long calls;                          /* calls to the slow function */
static int  audit_log[N], audit_n;

static int verify_card(const Passenger *x)   /* pretend this is slow */
{
    calls++;
    return x->card;
}
static int verify_card_audit(const Passenger *x, int id)
{
    audit_log[audit_n++] = id;               /* audit: record every passenger checked */
    return verify_card(x);
}

static int trace(const char *name, int v) { printf("  %s evaluated\n", name); return v; }

int main(void)
{
    printf("Short-circuit demo:\n");
    printf(" 0 && f():\n"); (void)(trace("left", 0) && trace("right", 1));
    printf(" 1 || f():\n"); (void)(trace("left", 1) || trace("right", 1));
    printf(" 0 &  f():\n"); (void)(trace("left", 0) &  trace("right", 1));
    printf("\n");
    srand(42);
    for (int i = 0; i < N; i++) {
        p[i].senior   = rand() % 100 < 20;   /* 20% senior citizens        */
        p[i].eligible = rand() % 100 < 60;   /* 60% in an eligible class   */
        p[i].card     = rand() % 100 < 10;   /* 10% hold a disability card */
    }
    int granted;

    /* A: cheap test first, slow call last (short-circuit ||) */
    calls = 0; granted = 0;
    for (int i = 0; i < N; i++)
        if ((p[i].senior && p[i].eligible) || verify_card(&p[i])) granted++;
    printf("A (senior&&eligible) || verify : calls=%4ld granted=%d\n", calls, granted);

    /* B: slow call first */
    calls = 0; granted = 0;
    for (int i = 0; i < N; i++)
        if (verify_card(&p[i]) || (p[i].senior && p[i].eligible)) granted++;
    printf("B verify || (senior&&eligible) : calls=%4ld granted=%d\n", calls, granted);

    /* C: non-short-circuit | and & : both sides always evaluated */
    calls = 0; granted = 0;
    for (int i = 0; i < N; i++)
        if (((p[i].senior & p[i].eligible) | verify_card(&p[i]))) granted++;
    printf("C (senior&eligible) | verify   : calls=%4ld granted=%d\n", calls, granted);

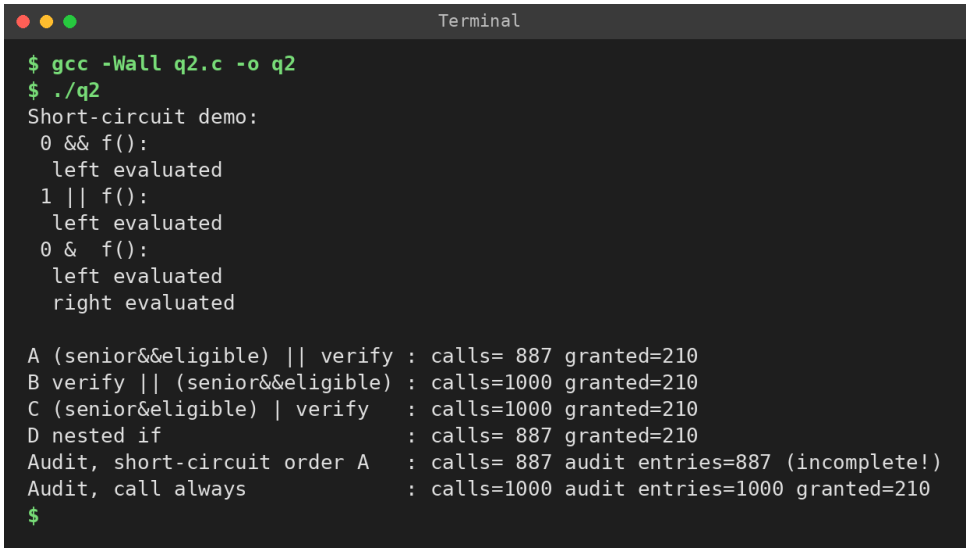
    /* D: nested ifs - same call count as A */
    calls = 0; granted = 0;
    for (int i = 0; i < N; i++) {
        if (p[i].senior) { if (p[i].eligible) { granted++; continue; } }
        if (verify_card(&p[i])) granted++;
    }
    printf("D nested if                    : calls=%4ld granted=%d\n", calls, granted);

    /* Changed condition: audit log must contain every passenger */
    calls = 0; audit_n = 0; granted = 0;
    for (int i = 0; i < N; i++)
        if ((p[i].senior && p[i].eligible) || verify_card_audit(&p[i], i)) granted++;
    printf("Audit, short-circuit order A   : calls=%4ld audit entries=%d (incomplete!)\n", calls, audit_n);

    calls = 0; audit_n = 0; granted = 0;
    for (int i = 0; i < N; i++) {
        int card = verify_card_audit(&p[i], i);      /* always called */
        if ((p[i].senior && p[i].eligible) || card) granted++;
    }
    printf("Audit, call always             : calls=%4ld audit entries=%d granted=%d\n", calls, audit_n, granted);
    return 0;
```

```
}
```

Output Screenshot



Analysis / Discussion

Experiment (a) and (c). In the demo, `0 && f()` and `1 || f()` printed only “left evaluated”; the right operand was never called. `0 & f()` printed both lines, so `&` evaluates both operands even though the left one is already 0.

Version	Slow calls (n = 1000)	Granted	Remark
A: (senior && eligible) || verify	887	210	113 passengers skipped the call
B: verify || (senior && eligible)	1000	210	slow call always first
C: (senior & eligible) | verify	1000	210	no short-circuit
D: nested if	887	210	same as A

All versions give the same answer (210 concessions) but differ in cost: order A saves 113 calls (11.3%) because 113 passengers were already decided by the cheap test. Order B and the `&`/`|` version always call the slow function.

When should the cheapest / most-likely-false condition be first? For `&&` put the cheapest or most-likely-FALSE condition first; for `||` put the cheapest or most-likely-TRUE condition first, and keep the slow, expensive check last. The order is irrelevant when both conditions are cheap and free of side effects, or when both must always be evaluated anyway. Order never changes the logical result of a side-effect-free condition, only the cost.

Changed condition (audit log). With order A only 887 passengers were logged, so 113 passengers are missing from the audit trail; the output shows “incomplete!”. If the audit must contain every passenger, the verification must be called for every passenger regardless of the order (1000 calls, 1000 log entries) – the call should be moved out of the condition into its own statement (`int card = verify(...)`). Then the order of the conditions no longer affects the call count. If the audit only needs to cover those actually checked, order A can stay. I recommend the first option, because a requirement for a complete audit is more important than saving calls, and hiding a required side effect inside a short-circuit operand is fragile.

Alternative concept – nested ifs or `&` / `|`. Nested `if` statements behave exactly like the short-circuit operators: the slow call is made only for passengers not decided earlier (887 calls). The non-short-circuit operators always evaluate both sides, so the slow function is called n times. For n passengers: short-circuit / nested-if makes n − k calls (k = passengers already decided by the cheap test), `&` and `|` make exactly n calls. All are O(n) in the worst case, space O(1); the difference is the constant factor, which matters because the slow call takes a network round trip.

Result

Short-circuit evaluation was demonstrated and measured: placing the cheap condition first reduced slow calls from 1000 to 887 with identical results, and `&`/`|` gave no saving. For the audit-log requirement the call must be unconditional, making order irrelevant. I learned that conditions with side effects must not depend on short-circuiting.

TASK 3 — Engineer a Device-Status Register with Bitwise Operators

Task / Problem Statement

A low-cost smart-home controller has only a few bytes of memory and must remember whether each of up to 8 devices (light, fan, AC, heater, pump…) is ON or OFF. All states are stored in one `unsigned char`, one bit per device. Using only bitwise operators, write functions to switch a device on, switch it off, toggle it, test whether it is on, print the register in binary and count the devices that are on. Predict the register value after a sequence of operations before running, explain why `reg & 1 << dev` and `(reg & 1) << dev` differ, explain what goes wrong with a signed char when bit 7 is set, and complete TODO 1 to TODO 4.

Aim

To manipulate individual bits of a status register with `|`, `&`, `^`, `~` and shifts, and to understand operator precedence and signedness problems in bitwise code.

Concept / Theory

A byte has 8 bits, so bit number `dev` can hold the state of device `dev`. The mask `1u << dev` has only that bit set. **Set** a bit with OR (`reg | mask`), **clear** it with AND of the inverted mask (`reg & ~mask`), **toggle** with XOR (`reg ^ mask`), and **test** it by shifting right and masking with 1 (`(reg >> dev) & 1`). To count the devices that are on, `reg &= reg - 1` removes the lowest set bit, so the number of repetitions equals the number of ON devices.

In C the shift operators have higher precedence than `&`, `^` and `|`, and values smaller than `int` are promoted to `int` before the operation. For a `signed char` with bit 7 set, the promotion sign-extends the value (0x80 becomes −128 = 0xFFFFFF80) and a right shift of a negative number is implementation-defined, usually an arithmetic shift that copies the sign bit.

Procedure / Algorithm

1. Define the device bit numbers (LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4) and build the mask `1u << dev`.
2. TODO 1: `switch_on` returns `reg | (1u << dev)`; TODO 2: `switch_off` returns `reg & ~(1u << dev)`.
3. TODO 3: `toggle` returns `reg ^ (1u << dev)`; TODO 4: `is_on` returns `(reg >> dev) & 1u`.
4. Write `show()` to print the 8 bits from bit 7 down to bit 0, and `count_on()` to count the set bits.
5. Predict the register after each statement in `main`, run the program and compare; then test the precedence and signedness examples.

Program / Source Code

```
#include <stdio.h>

/* Each bit of one byte stores the ON/OFF state of one device. */
enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

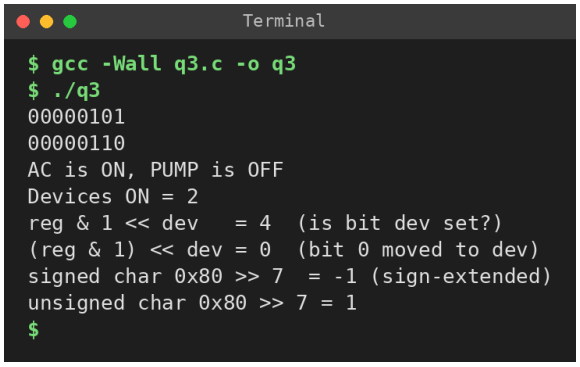
unsigned char switch_on(unsigned char reg, int dev)
{
    return reg | (1u << dev);                  /* TODO 1 */
}
unsigned char switch_off(unsigned char reg, int dev)
{
    return reg & ~(1u << dev);                 /* TODO 2 */
}
unsigned char toggle(unsigned char reg, int dev)
{
    return reg ^ (1u << dev);                  /* TODO 3 */
}
int is_on(unsigned char reg, int dev)
{
    return (reg >> dev) & 1u;                  /* TODO 4 */
}
void show(unsigned char reg)
{
    for (int bit = 7; bit >= 0; bit--) putchar(((reg >> bit) & 1u) ? '1' : '0');
    putchar('\n');
}
int count_on(unsigned char reg)               /* devices that are ON */
{
    int c = 0;
    while (reg) { reg &= reg - 1; c++; }       /* clears lowest set bit */
    return c;
}

int main(void)
{
    unsigned char reg = 0;
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);
    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);
    printf("AC is %s, PUMP is %s\n", is_on(reg, AC) ? "ON" : "OFF",
           is_on(reg, PUMP) ? "ON" : "OFF");
    printf("Devices ON = %d\n", count_on(reg));

    /* Inquiry: precedence of << versus & */
    unsigned char r = 0x04; int dev = 2;      /* only AC is on */
    printf("reg & 1 << dev   = %d  (is bit dev set?)\n", r & 1 << dev);
    printf("(reg & 1) << dev = %d  (bit 0 moved to dev)\n", (r & 1) << dev);

    /* Inquiry: signed char with bit 7 set */
    signed char s = (signed char)0x80;         /* -128 */
    printf("signed char 0x80 >> 7  = %d (sign-extended)\n", s >> 7);
    unsigned char u = 0x80;
    printf("unsigned char 0x80 >> 7 = %d\n", u >> 7);
    return 0;
}
```

Output Screenshot



Analysis / Discussion

Prediction before execution (register in binary / decimal after each statement):

Statement	Predicted register	Observed
reg = 0	00000000 (0)	–
switch_on(LIGHT)	00000001 (1)	–
switch_on(AC); show()	00000101 (5)	00000101 (match)
toggle(FAN)	00000111 (7)	–
switch_off(LIGHT); show()	00000110 (6)	00000110 (match)

The output matches the expected output: `00000101`, `00000110`, `AC is ON, PUMP is OFF`, and the count function reports 2 devices ON (FAN and AC).

Inquiry – precedence. `reg & 1 << dev` is parsed as `reg & (1 << dev)` because `<<` is higher than `&`; it tests bit `dev` and is non-zero when the device is on (for reg = 0x04, dev = 2 it gave 4). `(reg & 1) << dev` first keeps only bit 0 (the LIGHT bit) and then shifts it to position `dev`; it tells nothing about bit `dev` (it gave 0, even though AC is ON). The brackets change the meaning completely.

Inquiry – signed char. If the register is a `signed char`, bit 7 is the sign bit. When bit 7 is set the value is negative, it is promoted to a negative `int`, and `>> 7` gives −1 instead of 1 (the program printed −1 for the signed 0x80 and 1 for the unsigned one). Comparisons like `== 1` fail, printing shows negative numbers and the behaviour is implementation-defined. Hence the register must be `unsigned char`.

Alternative concept – `char state[8]`**.** Each operation (set, clear, toggle, test) is O(1) with an array just as with bits. Space: 8 bytes per home instead of 1 byte, so for 1,000,000 homes the array needs 8 MB while the bit register needs 1 MB. Counting the devices that are ON takes a loop over 8 elements (O(n) for n devices) in the array version, whereas the bit version loops only as many times as there are set bits (or uses a 256-entry lookup / hardware popcount in O(1)). The array is simpler to read, the bit register is 8 times smaller.

Result

All four bit functions were completed using only bitwise operators and the program produced the expected output (00000101, 00000110, AC is ON, PUMP is OFF); the predicted register values were correct at every step. I learned the set/clear/toggle/test mask patterns, that `<<` binds tighter than `&`, and that bit registers must be unsigned.

TASK 4 — else-if Ladder or switch?

Task / Problem Statement

The examination cell converts marks to grades (S, A, B, C, D, F) and then decides the scholarship category from the grade and family income. Marks outside 0–100 must be rejected. Rules: 90–100 S, 80–89 A, 70–79 B, 60–69 C, 50–59 D, below 50 F, invalid `'?'`. Implement the grade decision once with an else-if ladder and once with `switch` on `marks / 10`; find which inputs each version handles correctly (test 100, 90, 89, 0, −1, 101), the effect of a missing `break`, why `switch` cannot test ranges such as 75–84, which version is easier to change, design a boundary test set, choose the best structure for the scholarship rule, and complete TODO 1 with a switch.

Aim

To compare an else-if ladder with a switch statement for a range-based decision, find their boundary errors, and choose a suitable structure for a two-factor (grade and income) rule.

Concept / Theory

An **else-if ladder** tests conditions in order and runs the first one that is true, so it can express ranges (`marks >= 90`). A **switch** jumps directly to the `case` label equal to an integer expression; each label must be a single integer constant, not a range. Execution continues into the next case (**fall-through**) until a `break` or `return` is reached. Dividing the marks by 10 maps each 10-mark band to one value, and C integer division truncates toward zero, which also means `−5 / 10 = 0` and `101 / 10 = 10`, so out-of-range marks must be rejected before the switch.

Procedure / Algorithm

1. Reject marks below 0 or above 100 and return `'?'`.
2. Ladder version: test `>= 90`, `>= 80`, `>= 70`, `>= 60`, `>= 50` in that order and return the first matching grade, else `F`.
3. Switch version (TODO 1): `switch (marks / 10)` with `case 10: case 9:` returning S, then 8 → A, 7 → B, 6 → C, 5 → D, `default` → F.
4. Run the six boundary inputs through the ladder, the switch and a switch without the range check; then remove one `return`/`break` and note the change.
5. Build the scholarship decision as a lookup table indexed by grade and income band.

Program / Source Code

```
#include <stdio.h>

char grade_ladder(int m)
{
    if (m < 0 || m > 100) return '?';
    else if (m >= 90) return 'S';
    else if (m >= 80) return 'A';
    else if (m >= 70) return 'B';
    else if (m >= 60) return 'C';
    else if (m >= 50) return 'D';
    else return 'F';
}

char grade(int marks)
{
    if (marks < 0 || marks > 100) return '?';
    switch (marks / 10) {                   /* TODO 1 */
        case 10:
        case 9:  return 'S';
        case 8:  return 'A';
        case 7:  return 'B';
        case 6:  return 'C';
        case 5:  return 'D';
        default: return 'F';
    }
}

char grade_nocheck(int marks)               /* switch WITHOUT range check */
{
    switch (marks / 10) {
        case 10: case 9: return 'S';
        case 8:  return 'A';
        case 7:  return 'B';
        case 6:  return 'C';
        case 5:  return 'D';
        default: return 'F';
    }
}

char grade_missing_return(int marks)        /* 'return' removed from case 9 */
{
    char g = 'F';
    switch (marks / 10) {
        case 10: case 9: g = 'S';           /* break missing -> falls through */
        case 8:  g = 'A'; break;
        case 7:  g = 'B'; break;
        default: g = 'F';
    }
    return g;
}

int main(void)
{
    int marks[] = {100, 95, 83, 50, 49, -5};
    for (int i = 0; i < 6; i++) printf("%d:%c ", marks[i], grade(marks[i]));
    printf("\n\nBoundary tests (ladder / switch / switch without range check)\n");
    int t[] = {100, 90, 89, 0, -1, 101};
    for (int i = 0; i < 6; i++)
        printf("%4d : %c  %c  %c\n", t[i], grade_ladder(t[i]), grade(t[i]), grade_nocheck(t[i]));

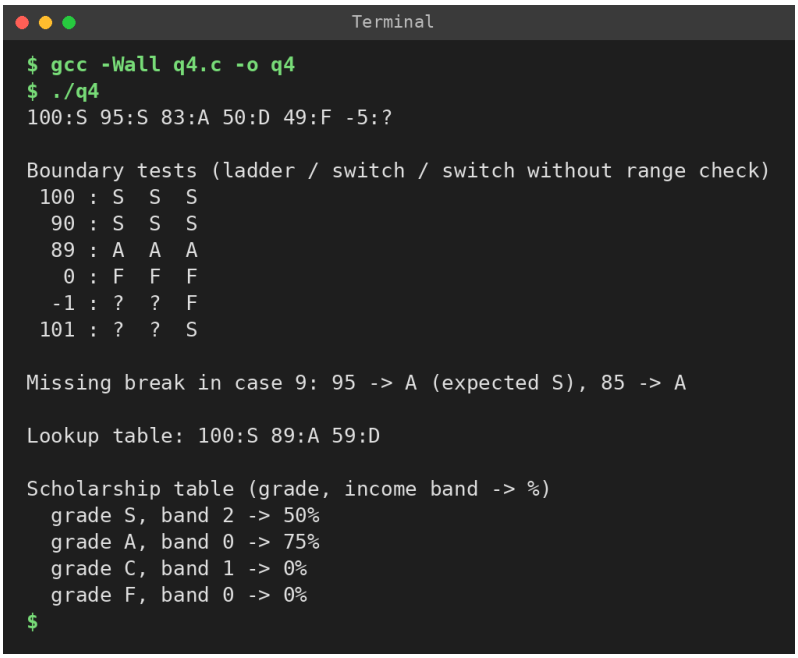
    printf("\nMissing break in case 9: 95 -> %c (expected S), 85 -> %c\n",
           grade_missing_return(95), grade_missing_return(85));

    /* lookup-table version */
    static const char table[11] = {'F','F','F','F','F','D','C','B','A','S','S'};
    printf("\nLookup table: 100:%c 89:%c 59:%c\n",
           table[100/10], table[89/10], table[59/10]);

    /* Design decision: scholarship % from grade (row) and income band (column) */
    const char *letters = "SABCDF";
    static const int pct[6][3] = {{100,75,50},{75,50,25},{50,25,0},{25,0,0},{0,0,0},{0,0,0}};
    const char tg[] = {'S','A','C','F'}; const int band[] = {2, 0, 1, 0};
    printf("\nScholarship table (grade, income band -> %%)\n");
    for (int i = 0; i < 4; i++) {
        int row = 0; while (letters[row] != tg[i]) row++;
        printf("  grade %c, band %d -> %d%%\n", tg[i], band[i], pct[row][band[i]]);
```

```
    }
    return 0;
}
```

Output Screenshot



Analysis / Discussion

Output of the completed program: `100:S 95:S 83:A 50:D 49:F -5:?` – identical to the expected output.

Input	Ladder	Switch (with check)	Switch (no check)
100	S	S	S
90	S	S	S
89	A	A	A
0	F	F	F
-1	?	?	F (wrong)
101	?	?	S (wrong)

With the range check in front, both versions are correct for all six inputs. Without it the switch gives wrong grades for −1 (−1/10 = 0 → F) and 101 (101/10 = 10 → S); the range check, not the switch, is what rejects invalid marks.

Missing break / return. After removing the `return` from case 9, marks 95 produced `A` instead of `S`, because execution fell through into `case 8`. (GCC with `-Wextra` also warns “this statement may fall through”.) Combining `case 10: case 9:` with nothing between them is intentional fall-through and is safe.

Why switch cannot test 75–84: a `case` label must be one constant, and 75–84 crosses two different values of `marks / 10` (7 and 8), so it would need ten separate labels or a different expression. A ladder handles it with `m >= 75 && m <= 84`. **Which is easier to change:** the ladder – changing a boundary is one number; the switch only works if all boundaries are multiples of 10, otherwise it must be rewritten.

Boundary test set: every boundary and its neighbour – 100, 101, 90, 89, 80, 79, 70, 69, 60, 59, 50, 49, 0, −1 – which exposes off-by-one errors (> instead of >=) and the truncation problem for negatives.

Design decision – scholarship rule. The rule depends on grade and income band, which is a 2-D table (6 grades × 3 bands). Nested ifs need 18 branches and a switch inside an if needs a case list per band; both mix data with logic and are error-prone. I choose a **lookup table** `pct[grade][band]`: one index operation, the rule can be read and audited as data, and changing a scholarship percentage does not touch any code. The example run printed e.g. grade S, band 2 → 50% (the percentages in the table are illustrative values chosen by me).

Alternative concept – lookup table / ladder. A table `char grades[11]` indexed by `marks / 10` finds the grade in O(1) time with one array access and 11 bytes of memory (the index must be range-checked first). The ladder needs up to k = 5 comparisons for an F grade (O(k)) and no extra memory; a switch is usually compiled to a jump table, so it is O(1) too.

Result

Both the else-if ladder and the switch version produced the correct grades for all valid and invalid boundary inputs once the range check was placed first; the missing-return experiment showed fall-through. I learned that the ladder is better for arbitrary ranges, the switch is best for equal-width bands, and a table is best when a decision depends on two factors.

TASK 5 — Count the Work Done by a Loop

Task / Problem Statement

A college fest awards prizes to tickets whose numbers are prime. The first program tests every divisor from 2 to n − 1, and tickets up to 1,000,003 will be verified. Starting only from the principle “if n has a divisor greater than √n, it must also have a divisor smaller than √n”, convert it into a loop condition without calling `sqrt()`, decide where `break`/`return` should stop the loop, count the divisions, implement both versions with `for` and the improved loop again with `while` and `do-while`, record the divisions for at least four ticket numbers including a large prime, and complete TODO 1.

Aim

To derive an efficient primality-test loop from a mathematical principle, count the work done by each loop version, and understand the boundary cases of `do-while` and integer overflow.

Concept / Theory

A number n is composite only if it has a divisor d with 1 < d ≤ √n (if d > √n divides n, then n/d < √n also divides it). So it is enough to try i = 2, 3, 4… while `i * i <= n`, which needs no `sqrt()` and no floating point. The loop can stop as soon as a divisor is found (`return 0`). `for` and `while` test the condition before the first pass, but `do-while` executes the body once before testing, so its first division happens even if the condition would be false. A global counter `checks` incremented before each `n % i` measures the work.

Procedure / Algorithm

1. Start with `checks = 0`; return 0 if n < 2.
2. Write the loop condition from the principle: `for (i = 2; i * i <= n; i++)`.
3. Inside the loop add 1 to `checks`, then `if (n % i == 0) return 0;` to stop at the first divisor.
4. After the loop return 1. Repeat with `while` and with `do-while` (adding a guard for n = 2 and n = 3).
5. Run for 97, 100, 7919 and 1000003; change the condition to `i < n` and record the new counts.

Program / Source Code

```
#include <stdio.h>

unsigned long checks = 0;   /* number of divisions performed by the last call */

/* Version 1: for loop, 2 .. n-1 */
int is_prime_slow(unsigned long n)
{
    checks = 0;
    if (n < 2) return 0;
    for (unsigned long i = 2; i < n; i++) {
        checks++;
        if (n % i == 0) return 0;
    }
    return 1;
}

/* Version 2: for loop, i * i <= n  (TODO 1) */
int is_prime(unsigned long n)
{
    checks = 0;
    if (n < 2) return 0;
    for (unsigned long i = 2; i * i <= n; i++) {
        checks++;
        if (n % i == 0) return 0;
    }
    return 1;
}

/* Version 3: while loop */
int is_prime_while(unsigned long n)
{
    checks = 0;
    if (n < 2) return 0;
    unsigned long i = 2;
    while (i * i <= n) {
        checks++;
        if (n % i == 0) return 0;
        i++;
    }
    return 1;
}

/* Version 4: do-while - needs a guard for n = 2, 3 */
int is_prime_dowhile(unsigned long n)
{
    checks = 0;
    if (n < 4) return n >= 2;      /* body would wrongly divide 2 by 2 */
    unsigned long i = 2;
    do {
        checks++;
        if (n % i == 0) return 0;
        i++;
    } while (i * i <= n);
    return 1;
}

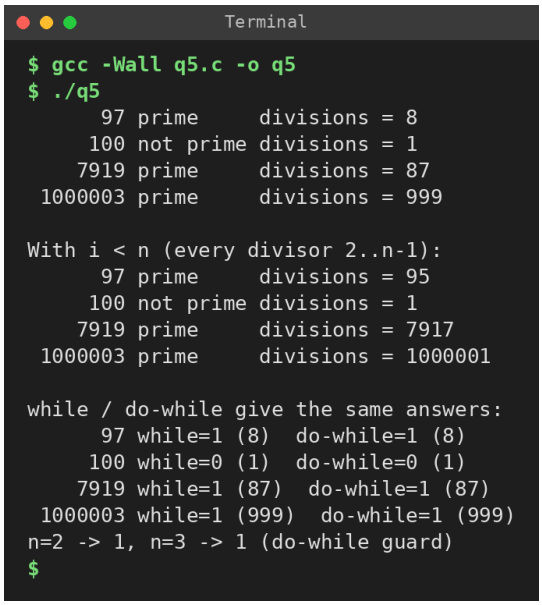
int main(void)
{
    unsigned long tickets[] = {97, 100, 7919, 1000003};
    for (int k = 0; k < 4; k++) {
        int p = is_prime(tickets[k]);
        printf("%8lu %-9s divisions = %lu\n", tickets[k], p ? "prime" : "not prime", checks);
    }

    printf("\nWith i < n (every divisor 2..n-1):\n");
    for (int k = 0; k < 4; k++) {
        int p = is_prime_slow(tickets[k]);
        printf("%8lu %-9s divisions = %lu\n", tickets[k], p ? "prime" : "not prime", checks);
    }

    printf("\nwhile / do-while give the same answers:\n");
    for (int k = 0; k < 4; k++) {
        int a = is_prime_while(tickets[k]);  unsigned long ca = checks;
        int b = is_prime_dowhile(tickets[k]); unsigned long cb = checks;
        printf("%8lu while=%d (%lu)  do-while=%d (%lu)\n", tickets[k], a, ca, b, cb);
    }
```

```
    printf("n=2 -> %d, n=3 -> %d (do-while guard)\n", is_prime_dowhile(2), is_prime_dowhile(3));
    return 0;
}
```

Output Screenshot



Analysis / Discussion

Observed divisions (the first four lines of the output match the expected output of the question):

Ticket n	Result	i * i <= n	i < n
97	prime	8	95
100	not prime	1	1
7919	prime	87	7917
1000003	prime	999	1000001

For a prime the improved loop does about √n divisions, 999 against 1,000,001 for the ticket 1000003, i.e. about 1000 times less work. For 100 both stop after one division because 2 divides it: the early `return` makes composites cheap, and the primes are the worst case. The `for`, `while` and `do-while` versions all gave the same results and the same counts.

Inquiry – do-while for n = 2 and n = 3. A `do-while` runs the body first. For n = 2 the first division is `2 % 2 == 0`, so 2 would be wrongly declared “not prime”, and for n = 3 it does one pointless division before the test `i * i <= n` is checked. A guard `if (n < 4) return n >= 2;` before the loop fixes both (the program printed 1 and 1).

Inquiry – overflow. For a very large `unsigned long n` (near 2^64) the value `i * i` can exceed the range once i is above about 4.29 × 10^9, so it wraps around to a small number and the condition `i * i <= n` stays true or becomes wrong. Writing the test as `i <= n / i` avoids multiplication and cannot overflow.

Alternative concept – Sieve of Eratosthenes. Marking composites once for all numbers up to N costs O(N log log N) time and O(N) memory (about 1 MB of flags for N = 1,000,003), after which every ticket is an O(1) lookup. Trial division needs O(√n) time per ticket and no extra memory. For N ≈ 10^6 the sieve costs roughly 2.6 million operations once, so it pays off when more than about 2,500 tickets must be verified; for a handful of tickets, or numbers too big for memory, trial division is better.

Result

The loop condition `i * i <= n` was derived from the principle, implemented with for, while and do-while, and the counted divisions (8, 1, 87, 999) matched the expected values; the naive version needed up to 1,000,001. I learned how to count the work of a loop, why do-while needs a guard, and why `i * i` can overflow.

TASK 6 — Loops That Must Terminate: Loan Repayment Schedule

Task / Problem Statement

A women's self-help group lends money at a monthly interest rate. Each month interest is added to the outstanding balance and the member repays a fixed amount. The program must print the month-by-month schedule until the loan is fully repaid and report the number of months and the total interest paid. Decide which loop matches the problem, the exact termination condition (and why `balance != 0` is dangerous with floating-point), what happens in the last month, and what happens if the repayment is smaller than the monthly interest. Verify with a small loan (Rs 10,000 at 2% per month, Rs 2,000 repayment) by hand, investigate how the iterations grow when the repayment decreases, add a 360-month safety limit and decide the message when it is reached.

Aim

To design a loop that is guaranteed to terminate, handle the last (partial) repayment correctly, avoid floating-point equality tests and protect against loans that can never be repaid.

Concept / Theory

A `while` loop checks its condition before every iteration, so it is the right choice when the number of repetitions is not known in advance and may even be zero (loan of 0). A `for` loop fits a known count and `do-while` forces at least one pass, so neither suits this problem. Floating-point numbers such as 0.02 cannot be stored exactly, so after many additions the balance may become 1e-13 instead of exactly 0, and the test `balance != 0` would never become false. A tolerance (here half a paisa, 0.005) must be used. A loop terminates only if each pass moves the variable toward the end condition: here only if the repayment is larger than that month's interest.

Procedure / Algorithm

1. Reject the loan at the start if `repayment <= balance * rate` (balance can never fall).
2. Use `while (balance > 0.005 && month < 360)`.
3. Each month: interest = balance × rate; balance += interest; paid = min(repayment, balance); balance −= paid; total interest += interest; month++; print the row.
4. After the loop report the months and the total interest, or the safety-limit message if the balance is still above 0.005.
5. Verify against the hand calculation, then run repayments from 2000 down to 201 to study the growth.

Program / Source Code

```
#include <stdio.h>
#include <math.h>

#define MAX_MONTHS 360

/* Prints schedule; returns months, or -1 if the loan can never be repaid */
int schedule(double balance, double rate, double pay, int verbose, double *tot_int)
{
    int month = 0;
    *tot_int = 0.0;
    if (pay <= balance * rate) return -1;           /* repayment never beats interest */
    if (verbose)
        printf("%-5s %-10s %-10s %-10s %-10s\n", "Month", "Opening", "Interest", "Paid", "Closing");
    while (balance > 0.005 && month < MAX_MONTHS) { /* never test balance != 0 */
        double opening  = balance;
        double interest = balance * rate;
        balance += interest;
        double paid = (pay < balance) ? pay : balance;  /* last month: pay only what is left */
        balance -= paid;
        *tot_int += interest;
        month++;
        if (verbose) printf("%-5d %-10.2f %-10.2f %-10.2f %-10.2f\n", month, opening, interest, paid, balance);
    }
    return (balance > 0.005) ? -2 : month;           /* -2: safety limit reached */
}

int main(void)
{
    double ti;
    int m = schedule(10000.0, 0.02, 2000.0, 1, &ti);
    printf("Months = %d, total interest = Rs %.2f\n\n", m, ti);

    printf("Growth: P=10000, r=2%%\n%-10s %-8s\n", "Repayment", "Months");
    double pays[] = {2000, 1000, 500, 300, 250, 210, 205, 201};
    for (int i = 0; i < 8; i++) {
        m = schedule(10000.0, 0.02, pays[i], 0, &ti);
        printf("%-10.0f %-8d\n", pays[i], m);
    }
    m = schedule(10000.0, 0.02, 200.0, 0, &ti);
    if (m == -1) printf("Repayment 200: REFUSED - repayment <= monthly interest, loan never ends\n");
    m = schedule(10000.0, 0.02, 200.1, 0, &ti);
    if (m == -2) printf("Repayment 200.1: stopped at %d months safety limit, balance still outstanding\n", MAX_MONTHS);

    double r = 0.02, P = 10000.0, E = 2000.0;
    printf("\nClosed form: n = %.4f months -> ceil = %.0f\n",
           -log(1 - r * P / E) / log(1 + r), ceil(-log(1 - r * P / E) / log(1 + r)));
    return 0;
}
```

Output Screenshot

```
Terminal
$ gcc -Wall q6.c -o q6
$ ./q6
Month Opening    Interest   Paid       Closing
1     10000.00   200.00     2000.00    8200.00
2     8200.00    164.00     2000.00    6364.00
3     6364.00    127.28     2000.00    4491.28
4     4491.28    89.83      2000.00    2581.11
5     2581.11    51.62      2000.00    632.73
6     632.73     12.65      645.38     0.00
Months = 6, total interest = Rs 645.38

Growth: P=10000, r=2%
Repayment  Months
2000       6
1000       12
500        26
300        56
250        82
210        154
205        188
201        268
Repayment 200: REFUSED - repayment <= monthly interest, loan never ends
Repayment 200.1: stopped at 360 months safety limit, balance still outstanding

Closed form: n = 5.3205 months -> ceil = 6
$
```

Analysis / Discussion

Hand calculation (10,000 at 2%, repay 2,000). Month 1: 10,000 + 200 = 10,200 − 2,000 = 8,200. Month 2: 8,200 + 164 = 8,364 − 2,000 = 6,364. Month 3: + 127.28 = 6,491.28 − 2,000 = 4,491.28. Month 4: + 89.83 = 4,581.11 − 2,000 = 2,581.11. Month 5: + 51.62 = 2,632.73 − 2,000 = 632.73. Month 6: + 12.65 = 645.38, and only 645.38 is paid, so the balance is 0.00. Total interest = 200 + 164 + 127.28 + 89.83 + 51.62 + 12.65 = Rs 645.38 in 6 months. The program output is identical.

Termination and last month. The loop ends when the balance is below half a paisa (or at the month limit). In the last month the payment is `min(repayment, balance)`, so the member pays 645.38 instead of 2,000 and the balance becomes exactly zero rather than negative. If the repayment is less than or equal to the monthly interest (e.g. 200 against 200 interest), the balance never decreases, so the program refuses the loan up front ("REFUSED").

Growth investigation. Prediction: the number of months grows slowly at first and then explodes as the repayment approaches the interest of Rs 200, because the principal falls by only (repayment − interest) per month. Measured:

Repayment (Rs)	2000	1000	500	300	250	210	205	201
Months	6	12	26	56	82	154	188	268

The prediction was right: going from 2000 to 500 adds 20 months, but 205 → 201 adds 80 months, and a repayment of 200.1 would need about 384 months, so the 360-month limit was reached in the program.

Deployment decision. The limit of 360 months (30 years) stops the loop even if some input slips through. When it is reached the program must not print a normal "fully repaid" report. It should print a clear message such as "Loan cannot be repaid within 360 months – increase the monthly repayment" and stop with an error status. I defend this because a never-ending loop would hang the counter or flood the printer, and a silent cut-off would give the member a wrong report. The early check (repayment ≤ interest) gives a faster and more precise message, and the limit is the safety net.

Alternative concept – closed formula. n = −log(1 − r·P / E) / log(1 + r) gives n = 5.3205 months, so 6 months after rounding up, in O(1) time instead of O(n) iterations; space is O(1) for both (the loop prints O(n) lines if the schedule is wanted). Precision problems of the formula: if E ≤ r·P the logarithm argument is ≤ 0 (NaN or error); rounding

errors can turn an exact integer such as 6.0000000001 into 7 with `ceil`; and it cannot give the schedule or the smaller final payment. The loop accumulates tiny rounding errors every month but gives the real schedule, so I use the formula only as a cross-check.

Result

The `while` loop with a tolerance condition and a 360-month limit produced a schedule that matches the hand calculation (6 months, total interest Rs 645.38), handled the smaller last payment, refused non-repayable loans and reported the limit case. I learned never to compare floating-point numbers with `!= 0` and always to guarantee that a loop moves toward its termination.

TASK 7 — Diagnose, Redesign and Validate

Task / Problem Statement

Review the evidence from Q1–Q6 and select ONE program that shows a significant limitation as the input size or requirement changes. The program must not simply be rewritten with another loop or decision statement: the problem has to be understood first and an appropriate redesign developed. Required parts: A. diagnose the bottleneck, B. define a measurable design objective, C. design the revised solution (with pseudocode), D. predict before implementing, E. implement and validate (original against redesign on the same inputs), F. defend the final design.

Selected program: the prime-ticket test of Task 5 (`is_prime` with the loop `i * i <= n`).

Aim

To diagnose where the prime test spends its divisions, set a measurable objective, redesign the algorithm by exploiting a mathematical property, and prove with the same inputs that the redesign is correct and cheaper.

Concept / Theory

The cost of trial division is the number of `n % i` operations. Divisors up to √n must be tried, but they do not all have to be tried: if n is not divisible by 2, it cannot be divisible by any other even number, because an even divisor would make n even. So after testing 2 only odd candidates 3, 5, 7… are needed. This is a change of algorithm that uses a property of numbers, not a change of loop statement; it keeps the O(√n) order but halves the constant.

Procedure / Algorithm

1. Take the evidence of Task 5: divisions for 97, 100, 7919 and 1000003.
2. Find which divisions are redundant (all even divisors after 2).
3. Redesign: test n = 2 and even n first with a single division by 2; then for (i = 3; i * i <= n; i += 2).
4. Predict the new division counts before coding: about half of the original for primes.
5. Run both versions on the same inputs, and on every n from 1 to 100,000, comparing results and total divisions.

Program / Source Code

```
#include <stdio.h>

unsigned long checks = 0;

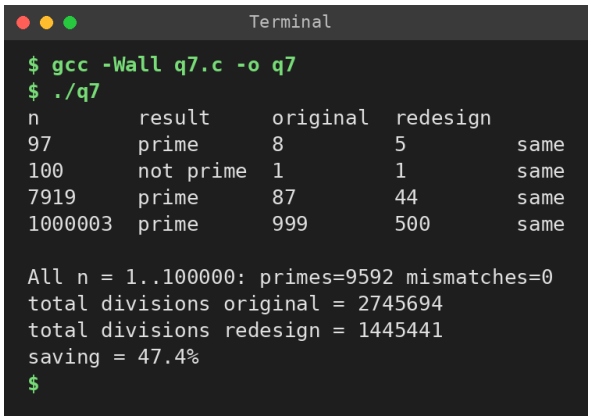
int is_prime_orig(unsigned long n)           /* original: every i from 2 */
{
    checks = 0;
    if (n < 2) return 0;
    for (unsigned long i = 2; i * i <= n; i++) { checks++; if (n % i == 0) return 0; }
    return 1;
}

int is_prime_odd(unsigned long n)            /* redesign: 2, then odd divisors only */
{
    checks = 0;
    if (n < 2) return 0;
    checks++; if (n % 2 == 0) return n == 2;
    for (unsigned long i = 3; i * i <= n; i += 2) { checks++; if (n % i == 0) return 0; }
    return 1;
}

int main(void)
{
    unsigned long t[] = {97, 100, 7919, 1000003};
    printf("%-8s %-10s %-9s %-9s\n", "n", "result", "original", "redesign");
    for (int k = 0; k < 4; k++) {
        int a = is_prime_orig(t[k]);  unsigned long ca = checks;
        int b = is_prime_odd(t[k]);   unsigned long cb = checks;
        printf("%-8lu %-10s %-9lu %-9lu %s\n", t[k], a ? "prime" : "not prime",
               ca, cb, a == b ? "same" : "MISMATCH");
    }

    unsigned long tot_a = 0, tot_b = 0, primes = 0; int bad = 0;
    for (unsigned long n = 1; n <= 100000; n++) {
        int a = is_prime_orig(n); tot_a += checks;
        int b = is_prime_odd(n);  tot_b += checks;
        if (a != b) bad++;
        primes += a;
    }
    printf("\nAll n = 1..100000: primes=%lu mismatches=%d\n", primes, bad);
    printf("total divisions original = %lu\ntotal divisions redesign = %lu\nsaving = %.1f%%\n",
           tot_a, tot_b, 100.0 * (tot_a - tot_b) / tot_a);
    return 0;
}
```

Output Screenshot



Analysis / Discussion

A. Diagnose the bottleneck (2 marks). Almost all work is the modulo operation `n % i` inside the loop, and its count grows with √n: 8 for 97, 87 for 7919 and 999 for 1000003 (Task 5). Work is clearly repeated unnecessarily: after `n % 2` has shown n is odd, the loop still tests i = 4, 6, 8… 1000, which can never divide an odd n. For the large prime 1000003, 499 of the 999 divisions (the even values 4, 6, …, 1000) are wasted. Composites such as 100 stop after one division, so primes are the worst case.

B. Design objective (2 marks). Reduce the number of divisions for prime tickets by about 50% (999 → about 500 for 1000003) with exactly the same true/false answer for every n, and without extra memory.

C. Revised solution (4 marks). *Property exploited:* an odd number has no even divisor. *Change:* divide by 2 once; for odd n, try only odd divisors i = 3, 5, 7… while i * i <= n. *Memory:* none (O(1)). *Trade-off:* one extra special case in the code (n = 2 and even n), a slightly longer function, still O(√n) time, only the constant is halved. Pseudocode:

```
is_prime_odd(n):
    if n < 2: return false
    count one division; if n mod 2 = 0: return (n = 2)
    i = 3
    while i * i <= n:
        count one division
        if n mod i = 0: return false
        i = i + 2
    return true
```

D. Prediction (1 mark). For 1000003 the odd divisors 3, 5, …, 999 are 499 values, plus the initial division by 2, so I predict about 500 divisions (≈ 50% of 999). Similarly 97 → 5 and 7919 → 44. Over a whole range the saving should be a little below 50% because composites exit early and gain less.

E. Implementation and validation (2 marks). Same inputs, same counter:

Ticket n	Result	Original divisions	Redesign divisions	Same answer
97	prime	8	5	yes
100	not prime	1	1	yes
7919	prime	87	44	yes
1000003	prime	999	500	yes

For all n from 1 to 100,000 (9,592 primes found by both) there were 0 mismatches; total divisions fell from 2,745,694 to 1,445,441, a saving of 47.4%. The predicted counts 5, 44 and 500 were exactly what was observed.

F. Defend the final design (2 marks). The objective was achieved: the worst case (large primes) needs 49.9% fewer divisions and the whole 1–100,000 range 47.4% fewer, with identical answers and no memory. The new cost is one extra special-case branch and slightly more code. The improvement is a constant factor of 2, not a better growth order: the work is still proportional to √n, so 500 divisions are needed for 1000003. The original may still be preferred for tiny inputs or when the simplest code matters, and for millions of queries the Sieve of Eratosthenes (Task 5) is better still, at the cost of O(N) memory. A further step, testing only 6k ± 1 candidates, would cut the work to about one third of the original.

Result

The bottleneck (redundant even divisors) was diagnosed from the evidence, a measurable 50% objective was set, the odd-divisor redesign was predicted to need 500 divisions for 1000003 and measured exactly 500, with identical results for all n up to 100,000 and 47.4% fewer divisions overall. I learned that a real improvement comes from using a property of the problem and from checking it with counted evidence, not from switching the loop statement.

Test Cases / Validation Table

Record different inputs, expected results, actual results, and whether each test passed.

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1 (Task 1)	price 999, qty 2, senior	Intended total 1783.21	Original 1798; fixed 1783.21	Original fails, fixed passes
2 (Task 2)	1000 passengers, order A vs B	Same 210 concessions, fewer slow calls	A: 887 calls, B: 1000 calls, both 210	Pass
3 (Task 3)	on LIGHT, on AC, toggle FAN, off LIGHT	00000101, 00000110, AC ON, PUMP OFF	00000101, 00000110, AC is ON, PUMP is OFF	Pass
4 (Task 4)	marks 100, 90, 89, 0, -1, 101	S, S, A, F, ?, ?	S, S, A, F, ?, ?	Pass
5 (Task 5)	n = 97, 100, 7919, 1000003	Divisions 8, 1, 87, 999	8, 1, 87, 999	Pass
6 (Task 6)	Rs 10,000, 2%, Rs 2,000 repayment	6 months, interest Rs 645.38	6 months, Rs 645.38	Pass
7 (Task 7)	n = 1 to 100,000, original vs redesign	Same answers, about 50% fewer divisions	0 mismatches, 47.4% fewer (1000003: 999 to 500)	Pass

Final Submission Checklist

☐ Student and course details completed

☑ Task statement, aim, theory, and algorithm included

☑ Program code is readable and runs successfully

☑ Output screenshot included

☑ Analysis and result written in own words

☑ Test cases completed where applicable

☐ GitHub repository link added if required